NVDA Stock Price Prediction (Next Three Days) using LSTM-NN &TensorFlow

AARON DE LA ROSA

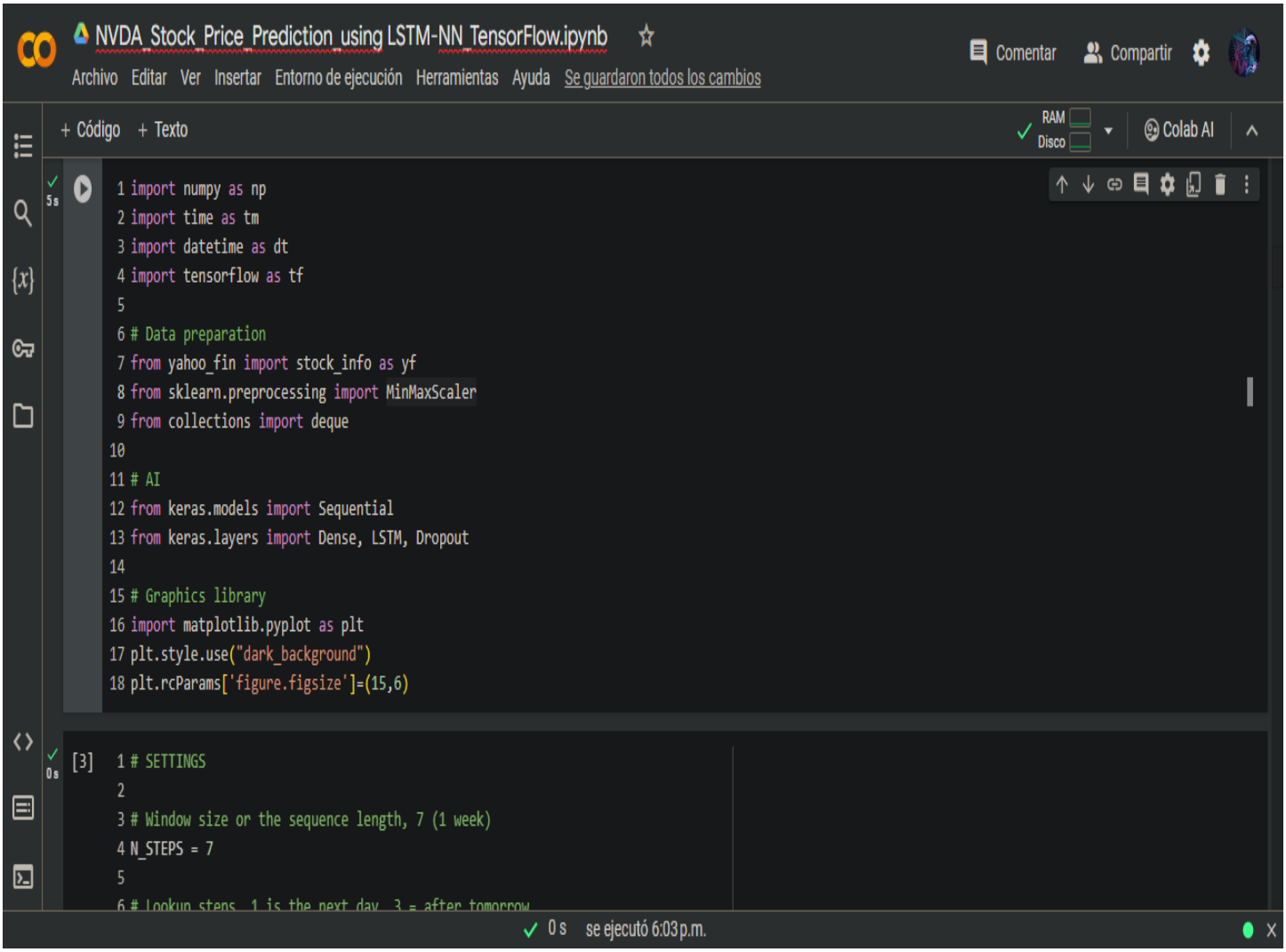
We are going to predict NVDA Stock for the next three days using Long Short-Term Memory (LSTM) networks and TensorFlow. LSTM's aim to provide a solution for maintaining long-term dependencies in sequential data, such as financial time series.

LSTMs is a powerful tool for handling sequential data, ensuring that relevant information is retained over extended periods. We'll use LSTM-NN for prediction based on our time series data for NVDA.

TensorFlow is a free and open-source software library for machine learning and artificial intelligence. It serves a wide range of tasks, with a particular focus on training and inference of deep neural networks.

We are going to combine these tools to predict the NVDA Stock Price for the next three days.

TensorFlow will empower our deep learning model effectively, making it a valuable tool in the field forecasting using artificial intelligence.



NVDA_Stock_Price_Prediction_using LSTM-NN_TensorFlow.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

[3]
```
# SETTINGS

# Window size or the sequence length, 7 (1 week)
N_STEPS = 7

# Lookup steps, 1 is the next day, 3 = after tomorrow
LOOKUP_STEPS = [1, 2, 3]

# Stock ticker, NVDA
STOCK = 'NVDA'

# Current date
date_now = tm.strftime('%Y-%m-%d')
date_3_years_back = (dt.date.today() - dt.timedelta(days=1104)).strftime('%Y-%m-%d')
```

[4]
```
# LOAD DATA
# from yahoo_fin
# for 1104 bars with interval = 1d (one day)
init_df = yf.get_data(
    STOCK,
    start_date=date_3_years_back,
    end_date=date_now,
    interval='1d')
```

0 s se ejecutó 6:03 p.m.

NVDA_Stock_Price_Prediction_using LSTM-NN_TensorFlow.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

```
    interval='1d')
```

```
init_df
```

	open	high	low	close	adjclose	volume	ticker
2021-01-29	130.750000	133.347504	129.115005	129.897507	129.605408	27155200	NVDA
2021-02-01	130.532501	132.707504	129.027496	132.369995	132.072342	21720400	NVDA
2021-02-02	133.987503	135.720001	132.854996	135.567505	135.262634	22044000	NVDA
2021-02-03	136.360001	139.317505	135.164993	135.304993	135.000732	24540800	NVDA
2021-02-04	135.309998	136.735001	133.377502	136.642502	136.335220	20134000	NVDA
...	...	...	...	...	...	...	...
2024-01-31	614.400024	622.690002	607.000000	615.270020	615.270020	45379500	NVDA
2024-02-01	621.000000	631.909973	616.500000	630.270020	630.270020	36914600	NVDA
2024-02-02	639.739990	666.000000	636.900024	661.599976	661.599976	47578000	NVDA
2024-02-05	682.250000	694.969971	672.049988	693.320007	693.320007	68007800	NVDA
2024-02-06	696.299988	697.539917	663.000000	682.229980	682.229980	67829294	NVDA

760 rows × 7 columns

0 s se ejecutó 6:03 p.m.

NVDA_Stock_Price_Prediction_using LSTM-NN_TensorFlow.ipynb

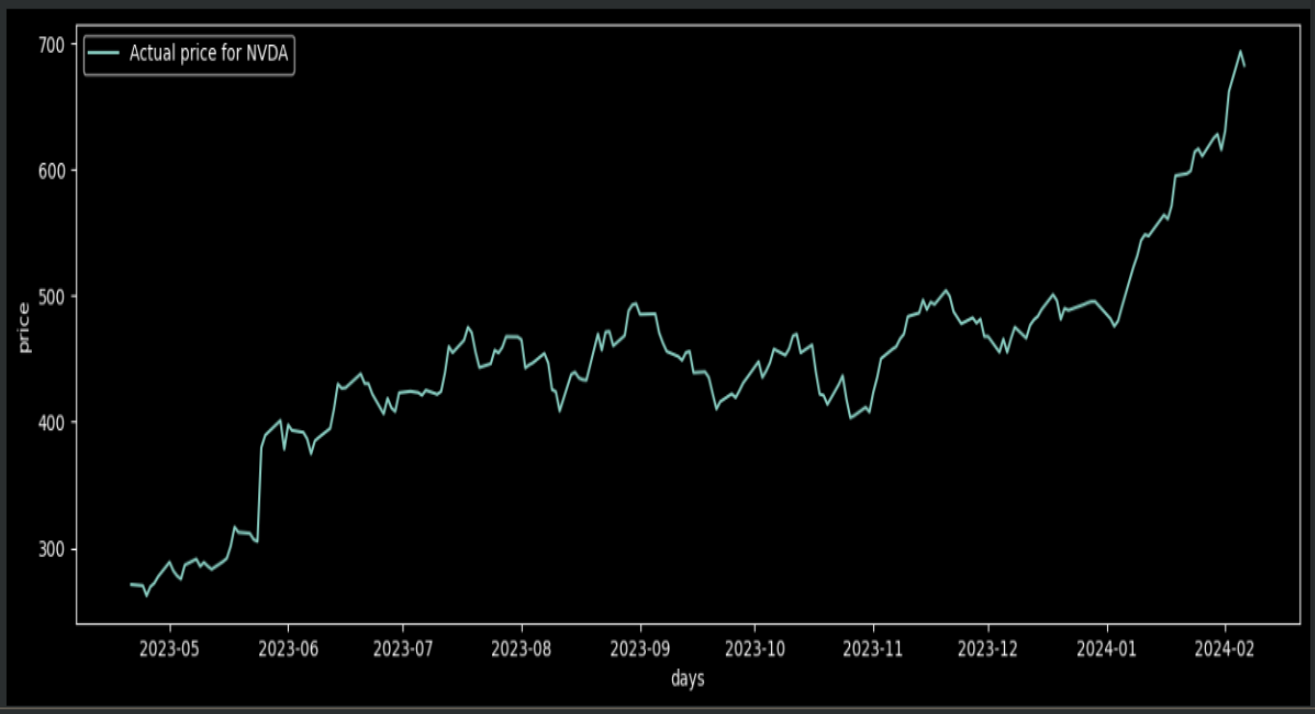
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

```
7 plt.legend([f'Actual price for {STOCK}'])
8 plt.show()
```



0 s se ejecutó 6:49 p.m.

NVDA_Stock_Price_Prediction_using LSTM-NN_TensorFlow.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

```
[9] 1 # Scale data for ML engine
    2 scaler = MinMaxScaler()
    3 init_df['scaled_close'] = scaler.fit_transform(np.expand_dims(init_df['close'].values, axis=1))
```

```
1 init_df
```

	close	date	scaled_close
2021-01-29	129.897507	2021-01-29	0.030337
2021-02-01	132.369995	2021-02-01	0.034593
2021-02-02	135.567505	2021-02-02	0.040096
2021-02-03	135.304993	2021-02-03	0.039644
2021-02-04	136.642502	2021-02-04	0.041946
...	...	...	...
2024-01-31	615.270020	2024-01-31	0.865674
2024-02-01	630.270020	2024-02-01	0.891490
2024-02-02	661.599976	2024-02-02	0.945409
2024-02-05	693.320007	2024-02-05	1.000000

0 s se ejecutó 6:49 p.m.

NVDA_Stock_Price_Prediction_using LSTM-NN_TensorFlow.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

[10]

	2024-02-05	693.320007	2024-02-05	1.000000
	2024-02-06	682.229980	2024-02-06	0.980914

760 rows × 3 columns

[11]

```
def PrepareData(days):
  df = init_df.copy()
  df['future'] = df['scaled_close'].shift(-days)
  last_sequence = np.array(df[['scaled_close']].tail(days))
  df.dropna(inplace=True)
  sequence_data = []
  sequences = deque(maxlen=N_STEPS)

  for entry, target in zip(df[['scaled_close'] + ['date']].values, df['future'].values):
      sequences.append(entry)
      if len(sequences) == N_STEPS:
          sequence_data.append([np.array(sequences), target])

  last_sequence = list([s[:len(['scaled_close'])] for s in sequences]) + list(last_sequence)
  last_sequence = np.array(last_sequence).astype(np.float32)

  # construct the X's and Y's
  X, Y = [], []
  for seq, target in sequence_data:
```

0 s se ejecutó 6:49 p.m.

NVDA_Stock_Price_Prediction_using LSTM-NN_TensorFlow.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

[11]

```
  df['future'] = df['scaled_close'].shift(-days)
  last_sequence = np.array(df[['scaled_close']].tail(days))
  df.dropna(inplace=True)
  sequence_data = []
  sequences = deque(maxlen=N_STEPS)

  for entry, target in zip(df[['scaled_close'] + ['date']].values, df['future'].values):
      sequences.append(entry)
      if len(sequences) == N_STEPS:
          sequence_data.append([np.array(sequences), target])

  last_sequence = list([s[:len(['scaled_close'])] for s in sequences]) + list(last_sequence)
  last_sequence = np.array(last_sequence).astype(np.float32)

  # construct the X's and Y's
  X, Y = [], []
  for seq, target in sequence_data:
      X.append(seq)
      Y.append(target)

  # convert to numpy arrays
  X = np.array(X)
  Y = np.array(Y)

  return df, last_sequence, X, Y
```

0 s se ejecutó 6:49 p.m.

NVDA_Stock_Price_Prediction_using_LSTM-NN_TensorFlow.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

[14]
```
# GET PREDICTIONS
predictions = []

for step in LOOKUP_STEPS:
  df, last_sequence, x_train, y_train = PrepareData(step)
  x_train = x_train[:, :, :len(['scaled_close'])].astype(np.float32)

  model = GetTrainedModel(x_train, y_train)

  last_sequence = last_sequence[-N_STEPS:]
  last_sequence = np.expand_dims(last_sequence, axis=0)
  prediction = model.predict(last_sequence)
  predicted_price = scaler.inverse_transform(prediction)[0][0]

  predictions.append(round(float(predicted_price), 2))
```

```
Epoch 63/80
94/94 [==============================] - 1s 6ms/step - loss: 0.0014
Epoch 64/80
94/94 [==============================] - 1s 6ms/step - loss: 0.0012
Epoch 65/80
94/94 [==============================] - 1s 6ms/step - loss: 0.0015
Epoch 66/80
94/94 [==============================] - 1s 6ms/step - loss: 0.0018
Epoch 67/80
94/94 [==============================] - 1s 6ms/step - loss: 0.0014
Epoch 68/80
```

0 s se ejecutó 6:49 p.m.

NVDA_Stock_Price_Prediction_using_LSTM-NN_TensorFlow.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

[14]
```
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
1/1 [==============================] - 1s 534ms/step
```

[15]
```
if bool(predictions) == True and len(predictions) > 0:
  predictions_list = [str(d)+'$' for d in predictions]
  predictions_str = ', '.join(predictions_list)
  message = f'{STOCK} prediction for upcoming 3 days ({predictions_str})'

  print(message)
```

```
NVDA prediction for upcoming 3 days (695.0$, 702.71$, 683.89$)
```

[16]
```
# Execute model for the whole history range
copy_df = init_df.copy()
y_predicted = model.predict(x_train)
y_predicted_transformed = np.squeeze(scaler.inverse_transform(y_predicted))
first_seq = scaler.inverse_transform(np.expand_dims(y_train[:6], axis=1))
last_seq = scaler.inverse_transform(np.expand_dims(y_train[-3:], axis=1))
y_predicted_transformed = np.append(first_seq, y_predicted_transformed)
y_predicted_transformed = np.append(y_predicted_transformed, last_seq)
copy_df[f'predicted_close'] = y_predicted_transformed
```

```
24/24 [==============================] - 0s 3ms/step
```

0 s se ejecutó 6:49 p.m.

NVDA_Stock_Price_Prediction_using LSTM-NN_TensorFlow.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

RAM Disco Colab AI

```
1 copy_df
```

	close	date	scaled_close	predicted_close
2021-01-29	129.897507	2021-01-29	0.030337	152.507507
2021-02-01	132.369995	2021-02-01	0.034593	149.612503
2021-02-02	135.567505	2021-02-02	0.040096	153.302505
2021-02-03	135.304993	2021-02-03	0.039644	149.059998
2021-02-04	136.642502	2021-02-04	0.041946	148.289993
...	...	...	...	...
2024-01-31	615.270020	2024-01-31	0.865674	625.044556
2024-02-01	630.270020	2024-02-01	0.891490	629.778748
2024-02-02	661.599976	2024-02-02	0.945409	661.599976
2024-02-05	693.320007	2024-02-05	1.000000	693.320007
2024-02-06	682.229980	2024-02-06	0.980914	682.229980

760 rows × 4 columns

0 s se ejecutó 6:49 p.m.

NVDA_Stock_Price_Prediction_using LSTM-NN_TensorFlow.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

Comentar Compartir

+ Código + Texto

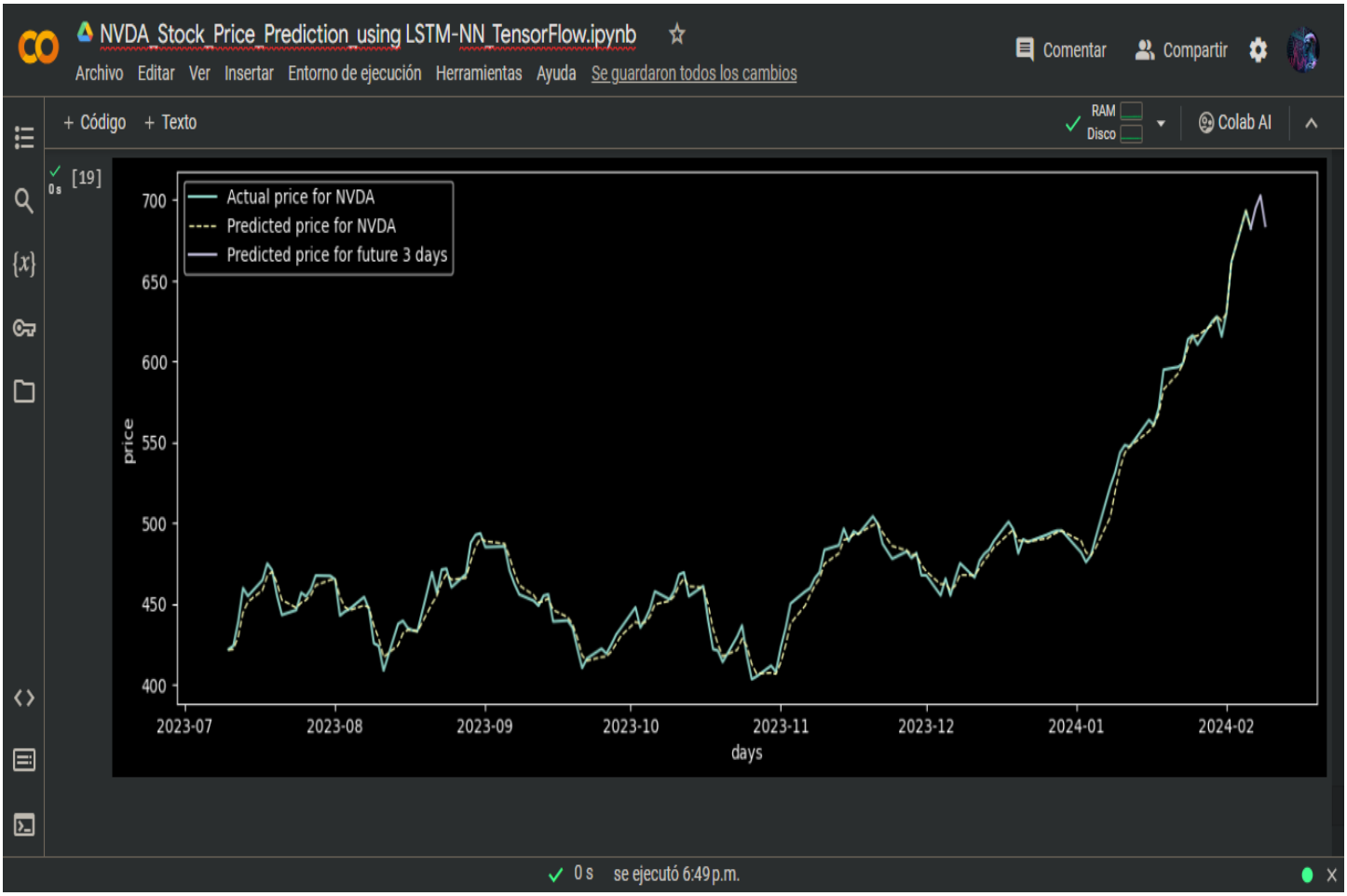
RAM Disco Colab AI

```
[18] 2 date_now = dt.date.today()
     3 date_tomorrow = dt.date.today() + dt.timedelta(days=1)
     4 date_after_tomorrow = dt.date.today() + dt.timedelta(days=2)
     5
     6 copy_df.loc[date_now] = [predictions[0], f'{date_now}', 0, 0]
     7 copy_df.loc[date_tomorrow] = [predictions[1], f'{date_tomorrow}', 0, 0]
     8 copy_df.loc[date_after_tomorrow] = [predictions[2], f'{date_after_tomorrow}', 0, 0]
```

```
<ipython-input-18-ca1349af6c77>:7: FutureWarning: Comparison of Timestamp with datetime.date is deprecated in order to match the standard library behavior
  copy_df.loc[date_tomorrow] = [predictions[1], f'{date_tomorrow}', 0, 0]
<ipython-input-18-ca1349af6c77>:8: FutureWarning: Comparison of Timestamp with datetime.date is deprecated in order to match the standard library behavior
  copy_df.loc[date_after_tomorrow] = [predictions[2], f'{date_after_tomorrow}', 0, 0]
```

```
[19] 1 # Result chart
     2 plt.style.use("dark_background")
     3 plt.figure(figsize=(15,5))
     4 plt.plot(copy_df['close'][-150:].head(147))
     5 plt.plot(copy_df['predicted_close'][-150:].head(147), linewidth=1, linestyle='dashed')
     6 plt.plot(copy_df['close'][-150:].tail(4))
     7 plt.xlabel('days')
     8 plt.ylabel('price')
     9 plt.legend([f'Actual price for {STOCK}',
    10            f'Predicted price for {STOCK}',
    11            f'Predicted price for future 3 days'])
```

0 s se ejecutó 6:49 p.m.



NVDA prediction for upcoming 3 days ($695.00, $702.71, $683.89).